

GUIA RESUMEN — Almacenamiento Local con Isar

Resumen integral para dominar Isar Community en ~20 minutos, sin volver a leer el modulo completo.

0 Flujo de aprendizaje general

3 PARTES • ~20 MIN

1 Fundamentos 6 min	2 Modelos y Operaciones 8 min	3 Implementacion 6 min
--	--	---

Regla de oro: "Isar es la base de datos local. Cada feature tiene su propia DataSource que maneja cache. Nunca consultes Supabase si el cache tiene datos frescos."

Parte	Que aprendes	Archivo clave
1. Fundamentos	Que es Isar, diferencias entre versiones, setup, code generation	01-isar-introduccion.md
2. Modelos y Operaciones	Colecciones, CRUD, queries, filtros, TTL, putAll, watch	02-modelos-operaciones.md
3. Implementacion	IsarService singleton, LocalDataSource, CacheManager	03-implementacion-local-datasource.md

1 Que es Isar? (3 min)

BASE DE DATOS EMBEBIDA

Isar es una base de datos NoSQL embebida para Flutter. Es rapida, tipada, y funciona sin servidor.

Las 3 versiones de Isar

Paquete	Estado	Version	Uso
isar	Abandonado	3.x	NO usar en proyectos nuevos
isar_community	Activo	3.x	Recomendado para la mayoría de proyectos
isar (v4)	En desarrollo	4.x	Rewrite en Rust, API diferente

Recomendacion: Usa isar_community para proyectos nuevos. Es el fork mantenido de la comunidad y es compatible con la API de Isar v3.

Instalacion

```
dependencies:
  isar_community: ^3.1.0+1
  isar_flutter_libs: ^3.1.0+1
  path_provider: ^2.1.0

dev_dependencies:
  isar_generator: ^3.1.0+1
  build_runner: ^2.4.0
```

Code Generation

```
# Generar archivos .g.dart
dart run build_runner build --delete-conflicting-outputs

# Watch para desarrollo continuo
dart run build_runner watch --delete-conflicting-outputs
```

Componentes clave

Componente	Que es	Ejemplo
Coleccion	Tabla de datos (como una tabla SQL)	CachedUser, CachedToken
Esquema	Clase generada por build_runner	CachedUserSchema
Indice	Acelera busquedas en campos especificos	@Index(unique: true)
Embedded	Objeto anidado dentro de una coleccion	PaymentInfo embebido en User

Ejemplo de coleccion

```
class CachedUser {
  Id id = Isar.autoIncrement; // ID auto-incremental

  @Index(unique: true)
  late String userId;          // ID del usuario en Supabase

  late String name;
  late String email;
  String? avatarUrl;
  bool isActive = true;

  late DateTime cachedAt;      // Cuando se guardo en cache
  late DateTime expiresAt;     // Cuando expira el cache
}
```

Patron TTL: Siempre guarda `cachedAt` y `expiresAt` en cada coleccion. Esto permite saber si el cache sigue siendo valido.

Operaciones basicas

```
// CREAR/ACTUALIZAR (put crea si no existe, actualiza si existe)
await isar.writeTxn(() async {
  await isar.cachedUsers.put(user);
});

// LEER por ID
final user = await isar.cachedUsers.get(userId);

// ELIMINAR por ID
await isar.writeTxn(() async {
  await isar.cachedUsers.delete(userId);
});

// ELIMINAR todos
await isar.writeTxn(() async {
  await isar.cachedUsers.where().deleteAll();
});
```

Regla de oro: writeTxn

Operacion	Requiere writeTxn?	Por que
put, delete, clear	Siempre	Escritura en disco
get, findAll, filter	No	Solo lectura

Error comun: Olvidar `writeTxn` al escribir. Isar lanza un error si intentas escribir sin transaccion.

where() — Búsqueda por índice

```
// Buscar por índice único
final user = await isar.cachedUsers
  .where()
  .userIdEqualTo(userId)
  .findFirst();

// Buscar todos los activos
final active = await isar.cachedUsers
  .where()
  .isActiveEqualTo(true)
  .findAll();
```

filter() — Filtros avanzados

```
// Filtros compuestos
final results = await isar.cachedUsers
  .filter()
  .isActiveEqualTo(true)
  .nameContains('Carlos')
  .findAll();

// Filtros con OR
final results = await isar.cachedUsers
  .filter()
  .nameContains('A')
  .or()
  .nameContains('B')
  .findAll();
```

Operadores disponibles

Operador	Ejemplo
EqualTo	.userIdEqualTo(id)
GreaterThan	.ageGreaterThan(18)
LessThan	.ageLessThan(65)
Contains	.nameContains('text')
Between	.ageBetween(18, 65)
IsNull / IsNotNull	.avatarUrlIsNull()

putAll() — Insercion batch

```
// Lento: N transacciones separadas
for (final user in users) {
  await isar.writeTxn(() => isar.cachedUsers.put(user));
}

// Rapido: 1 sola transaccion
await isar.writeTxn(() => isar.cachedUsers.putAll(users));
```

Metodo	1,000 objetos	Tiempo aprox.
put() en loop	1,000 txn	~500ms
putAll()	1 txn	~50ms

watch() — Streams reactivos

```
// Escuchar un solo objeto por ID
final stream = isar.cachedUsers.watchObject(userId);
stream.listen((user) {
  if (user != null) print('Actualizado: ${user.name}');
});

// Escuchar toda la coleccion
final stream = isar.cachedUsers.watchLazy();
stream.listen(() {
  print('La coleccion cambio');
});
```

Uso ideal: `watch()` mantiene la UI sincronizada con el cache sin llamar manualmente a `load()` despues de cada escritura.

6 IsarService — Singleton (2 min)

IsarService encapsula la inicializacion de Isar y se usa como singleton en toda la app.

Implementacion

```
class IsarService {
  static Isar? _instance;

  static Future<Isar> getInstance() async {
    if (_instance != null) return _instance!;
    final dir = await getApplicationDocumentsDirectory();
    _instance = await Isar.open(
      [CachedUserSchema, CachedTokenSchema, CachedProfileSchema],
      directory: dir.path,
    );
    return _instance!;
  }
}
```

Registro en DI

```
// En injection.dart
final isar = await IsarService.getInstance();
sl.registerLazySingleton(() => isar);
```

Importante: Abrir Isar una sola vez al inicio de la app. No crear multiples instancias.

Cada feature tiene su propia **LocalDataSource** que maneja el cache en Isar.

Estructura

```
abstract class AuthLocalDataSource {
    Future<void> cacheToken(String token);
    String? getCachedToken();
    Future<void> cacheUser(CachedUser user);
    CachedUser? getCachedUser();
    Future<void> clearCache();
}

class AuthLocalDataSourceImpl implements AuthLocalDataSource {
    final Isar _isar;
    AuthLocalDataSourceImpl(this._isar);

    @override
    Future<void> cacheToken(String token) async {
        await _isar.writeTxn(() async {
            await _isar.cachedTokens.clear();
            await _isar.cachedTokens.put(CachedToken()
                ..token = token
                ..cachedAt = DateTime.now()
                ..expiresAt = DateTime.now().add(Duration(hours: 2)));
        });
    }
    // ... demas metodos
}
```

CacheManager — Limpieza centralizada

```
class CacheManager {
    final _clearFns = <Future<void> Function()>[];

    void register(Future<void> Function() clearFn) {
        _clearFns.add(clearFn);
    }

    Future<void> clearAll() async {
        await Future.wait(_clearFns.map((fn) => fn()));
    }
}

// Uso al cerrar sesion
await cacheManager.clearAll();
await supabase.auth.signOut();
```

Regla: Cada feature registra su `clearCache()` en el `CacheManager`. Al cerrar sesion, se limpia todo de una vez.

El patron TTL permite guardar datos con una fecha de expiracion. Al leer, se verifica si el cache sigue valido.

Implementacion

```
// Guardar con TTL
await isar.writeTxn(() async {
  await isar.cachedUsers.put(CachedUser()
    ..userId = user.id
    ..name = user.name
    ..cachedAt = DateTime.now()
    ..expiresAt = DateTime.now().add(Duration(hours: 1)));
});

// Leer con validacion TTL
Future<CachedUser?> getCachedUser(String userId) async {
  final cached = await _isar.cachedUsers
    .where()
    .userIdEqualTo(userId)
    .findFirst();

  if (cached == null) return null;
  if (cached.expiresAt.isBefore(DateTime.now())) {
    return null; // Cache expirado
  }
  return cached;
}
```

Tiempos de cache recomendados

Datos	TTL sugerido	Razon
Token JWT	1-2 horas	Expiracion de Supabase
Perfil de usuario	30 min - 1 hora	Cambia poco
Metodos de pago	24 horas	Cambia muy raro
Lista de productos	5-15 minutos	Cambia frecuente

`UserSessionImpl` permite obtener el usuario autenticado actual de forma sincrona desde Isar, sin necesidad de hacer `await`.

Implementacion

```
class UserSessionImpl implements UserSession {
  final Isar _isar;

  UserSessionImpl(this._isar);

  @override
  String? get currentUserId {
    final user = _isar.cachedUsers
      .where()
      .findFirstSync();
    return user?.userId;
  }

  @override
  bool get isAuthenticated => currentUserId != null;
}
```

Uso: Inyecta `UserSession` en los Repositories que necesiten saber quien es el usuario actual (ej: `PaymentMethodRepository`).

- ✓ Instalar `isar_community` + `isar_flutter_libs` + `isar_generator`
- ✓ Crear colecciones con `@collection`, `@Index()`, `@enumerated`
- ✓ Ejecutar `build_runner build` para generar `.g.dart`
- ✓ Crear `IsarService` singleton con `Isar.open()`
- ✓ Registrar Isar en GetIt (DI)
- ✓ Crear `abstract class` + `Impl` para cada `LocalDataSource`
- ✓ Implementar `cacheX()`, `getCachedX()`, `clearCache()`
- ✓ Agregar patro TTL con `cachedAt` / `expiresAt`
- ✓ Usar `putAll()` para inserciones batch
- ✓ Usar `watch()` para UI reactiva
- ✓ Crear `CacheManager` y registrar `clearCache()` de cada feature
- ✓ Integrar `UserSessionImpl` para acceso sincrónico al usuario
- ✓ Manejar errores con `CacheException`

Tiempo de lectura del resumen: ~20 min · Isar es rápido, tipado, y perfecto para cache local en Flutter.